

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION RESEARCH I

Antibiotic-aware Adaptive Feature Fusion for Salmonella Antimicrobial Resistance Prediction

NGUYEN BICH LOAN

loan.nb235966@sis.hust.edu.vn

Student ID: 20235966

Program: Global ICT

Supervisor: Nguyen Hong Quang

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 2026

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my supervisor, Mr. Nguyen Hong Quang, for his guidance and feedback throughout this project, especially for encouraging critical, honest evaluation of results. I thank the faculty of the School of Information and Communications Technology for the knowledge that made this work possible. I am also grateful to my family and friends for their constant support during my studies. Finally, I thank the teams behind open resources such as BV-BRC, CARD and ResFinder, which made the external validation feasible. Any remaining shortcomings in this research are my own responsibility.

Nguyen Bich Loan

ABSTRACT

Antimicrobial resistance (AMR) in *Salmonella* is an urgent clinical and food-safety problem that calls for fast genome-based prediction tools. Existing studies typically train machine-learning models on genomic data with random data splits and report high accuracy, yet rarely control for three core risks: strong class imbalance, configuration selection based on the test set, and leakage from population structure. This research studies AMR prediction for *Salmonella* under an antibiotic-aware adaptive feature fusion approach, selecting a feature module and model for each antibiotic, while placing the reliability of the results at the centre. The solution is built on 1167 genomes with five original antibiotics and extended by external validation on BV-BRC for five further drugs. The research confirms that the accessory genome is the dominant predictive signal, clearly outperforming core SNPs. Adaptive feature fusion reaches high performance (F1 0.886–0.971) and improves the F1 score on 4 of the 5 drugs over the strong expert-marker baseline, with each antibiotic selecting a different best module — direct evidence that the best representation is antibiotic-specific. The research then validates these results more thoroughly than prior work: repeated cross-validation with an ordinary paired t-test; true phylogenetic splits (core SNPs internally, MLST externally) that identify which antibiotics are mechanism-driven and robust across lineages (AXO, FOX, TET) and which are more lineage-dependent; probability calibration and decision-curve analysis for screening; and real QRDR mutation calling (*gyrA*, *parC*) that provides lineage-robust, mechanism-based features for the quinolone group. The models are well-calibrated and useful for screening. The main contribution is a competitive, antibiotic-aware prediction framework together with a thorough reliability-checking protocol.

Student

Nguyen Bich Loan

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Background and Problems of Research	1
1.3 Research Objectives and Conceptual Framework	1
1.4 Contributions	2
1.5 Organization of Research.....	2
CHAPTER 2. LITERATURE REVIEW	3
2.1 Scope of Research	3
2.2 Related Work	3
2.3 Biological Background: Resistance Mechanisms and Genome Representation 3	
2.4 Machine-Learning Background: Models, Feature Selection and Evaluation 4	
CHAPTER 3. METHODOLOGY	5
3.1 Overview	5
3.2 Data and Preprocessing	5
3.3 Adaptive Feature Fusion.....	5
3.4 Anti-leakage and Evaluation Protocol	6
3.5 Statistical Testing.....	6
3.6 Lineage-aware Evaluation.....	6
3.7 Mutation Calling for Quinolones.....	6
CHAPTER 4. PROPOSED ARCHITECTURE AND PIPELINE	7
4.1 Design Principles and Overall Data Flow	7
4.2 Inputs and Partitioning	8
4.3 Feature Modules	8

4.4 Classifier, Threshold and Metrics	9
4.5 Adaptive Selection and Reliability Layer.....	9
4.6 Data File Formats and Examples	10
CHAPTER 5. IMPLEMENTATION AND SOURCE CODE.....	12
5.1 Data Loading and Preprocessing	12
5.2 Feature Selection	13
5.3 Feature Modules: Sample Graph and Gene Embedding	14
5.4 Model, Threshold and Fold Evaluation	15
5.5 Cross-validation Loop	16
5.6 Statistical Testing.....	17
5.7 Lineage-aware Clustering	18
5.8 External Data via the BV-BRC API	18
5.9 QRDR Mutation Calling	19
5.10 Summary	20
CHAPTER 6. THEORETICAL ANALYSIS	21
6.1 Paired t-Test on Cross-Validation Folds	21
6.2 Population-Structure Confounding	21
CHAPTER 7. NUMERICAL RESULTS.....	23
7.1 Evaluation Parameters.....	23
7.2 Simulation Method	23
7.3 RQ1: Accessory genome vs core SNP.....	23
7.4 RQ2: Adaptive fusion improves F1 on most drugs	24
7.5 RQ3: Drug metadata supports new-drug simulation	25
7.6 RQ4: Population structure and confounding	25
7.7 Single-marker reliance and biological interpretation	27
7.8 RQ5: Probability calibration and screening utility.....	28

7.9 RQ6: Mutation-level analysis for quinolones	28
7.10 Comparison with the original study	29
CHAPTER 8. CONCLUSIONS	30
8.1 Summary	30
8.2 Suggestion for Future Works.....	30
REFERENCE	32
APPENDIX.....	34
A. REPRODUCIBILITY AND ARTIFACTS.....	34
A.1 Data Sources	34
A.2 Notebook Catalogue.....	34
A.3 Environment and Execution	34
B. SUPPLEMENTARY RESULTS	35
B.1 Population Structure from Core SNPs	35
B.2 QRDR Mutation Association with Resistance	35

LIST OF FIGURES

Figure 4.1	Proposed architecture of the AMR prediction pipeline (page 1 of <code>architecture.drawio</code>).	7
Figure 4.2	Feature modules and output matrix sizes (page 2 of <code>architecture.drawio</code>)	
Figure 4.3	Reliability-checking layer with annotated matrix shapes (page 3 of <code>architecture.drawio</code>).	10
Figure 7.1	The accessory genome outperforms core SNPs across all five antibiotics.	24
Figure 7.2	Direction O adaptive fusion (best module per drug) versus the 50 expert markers: F1 improves on 4/5 drugs.	25
Figure 7.3	Original set: random F1 versus leave-sub-lineage-out (core SNP).	26
Figure 7.4	External: TET generalizes across lineages; CIP/NAL/GEN collapse toward baseline.	27
Figure 7.5	ECE decreases markedly after isotonic calibration.	28
Figure 7.6	Left: QRDR mutations are strongly associated with CIP resistance. Right: mutation features let NAL generalize across lineages.	29

LIST OF TABLES

Table 4.1	Notation and matrix dimensions ($m = 1167$ genomes, $g = 18125$ accessory genes).	8
Table 7.1	F1 of the accessory genome versus core SNPs (RQ1).	23
Table 7.2	Best configuration per antibiotic (Direction O adaptive fusion) with the expert-marker baseline. <i>Baseline F1</i> is the <code>paper_ready50</code> result (the 50 expert markers of the original 2025 study); <i>F1</i> , <i>Bal. acc.</i> and <i>AUPRC</i> are the adaptive-fusion result, and $\Delta F1$ is its gain over the baseline.	24
Table 7.3	External real-prevalence: random F1 versus MLST group-aware (RQ4).	26
Table 7.4	F1 when removing the top- k most important features (<code>accessory_chi2_200/LR</code>).	27
Table 7.5	Operating points for screening (LR, isotonic, threshold 0.5).	28
Table 7.6	What this research adds relative to the original study.	29
Table A.1	Main notebooks and their roles.	34
Table B.1	Population structure (core SNP clustering).	35
Table B.2	QRDR mutations and resistance rate (R-rate).	35

LIST OF ABBREVIATIONS

Abbreviation	Definition
AMR	Antimicrobial Resistance
API	Application Programming Interface
AUPRC	Area Under the Precision-Recall Curve
AUROC	Area Under the ROC Curve
BV-BRC	Bacterial and Viral Bioinformatics Resource Center
CARD	Comprehensive Antibiotic Resistance Database
CV	Cross-Validation
ECE	Expected Calibration Error
LR	Logistic Regression
MIC	Minimum Inhibitory Concentration
MLST	Multi-Locus Sequence Typing
NPV	Negative Predictive Value
PPV	Positive Predictive Value
QRDR	Quinolone Resistance-Determining Region
RF	Random Forest
SIR	Susceptible/Intermediate/Resistant phenotype classification
SNP	Single Nucleotide Polymorphism
XGB	Extreme Gradient Boosting

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

Genome-based prediction of antimicrobial resistance (Antimicrobial Resistance (AMR)) in *Salmonella* is the subject of this research, which builds on and critically re-evaluates the expert-marker prediction study of the original 2025 work [1]. AMR is one of the greatest threats to global public health, estimated to be associated with millions of deaths each year [2]. *Salmonella* is a common food-borne pathogen and a suitable subject for studying AMR because large volumes of genomic data with paired resistance phenotypes are available. The problem addressed here is: given the genomic data of a *Salmonella* strain, predict whether that strain is resistant or susceptible to a specific antibiotic.

Accurate and reliable genome-based AMR prediction can support rapid diagnosis, epidemiological surveillance and early warning, reducing reliance on time-consuming conventional susceptibility testing. However, the problem has three characteristic difficulties: the resistant class is often rare (some antibiotics have only about 6% resistant samples), the genomic feature space is very large (tens of thousands of genes), and the resistance mechanisms differ across antibiotics.

1.2 Background and Problems of Research

Many studies have applied machine learning to predict AMR from *Salmonella* genomes, using gene-based features (the accessory genome), point variants (core Single Nucleotide Polymorphism (SNP)), and Minimum Inhibitory Concentration (MIC) values [3]. Most report high accuracy on random data splits.

Nevertheless, current solutions share three important limitations. First, with strongly imbalanced data, reporting accuracy alone is misleading and is rarely accompanied by statistical significance testing. Second, when many feature/model configurations are tried and the best one is chosen by its score on the test set, the results are inflated by a “winner’s curse” effect. Third, bacterial genomes have strong population structure: strains of the same phylogenetic lineage are highly similar, so a random split may let the model learn lineage identity rather than the true resistance mechanism, giving an optimistic estimate of generalization [4].

1.3 Research Objectives and Conceptual Framework

The objective of this research is to build and *reliably evaluate* an AMR prediction framework for *Salmonella*. The research follows an antibiotic-aware adaptive feature fusion approach: instead of a fixed pipeline, each antibiotic is assigned a feature

module and model suited to its resistance mechanism.

Corresponding to the three limitations in Section 1.2, the research proposes: (i) rigorous statistical testing (a paired t-test on repeated cross-validation folds, with a Wilcoxon signed-rank test and a bootstrap confidence interval) to determine whether the advantage of adaptive fusion is real or noise; (ii) evaluation along true phylogenetic lineages (leave-lineage-out based on internal core SNPs and external Multi-Locus Sequence Typing (MLST)) to detect confounding; and (iii) true mechanism-level mutation calling (Quinolone Resistance-Determining Region (QRDR)) for the quinolone group to compare against annotation features.

1.4 Contributions

This research makes five main contributions:

1. It confirms that the accessory genome is the dominant AMR predictive signal, consistently outperforming core SNPs across the five antibiotics.
2. It proposes an antibiotic-aware adaptive feature fusion method that improves F1 on 4 of 5 drugs over the strong expert-marker baseline, with each antibiotic selecting a different best module, and validates this with repeated cross-validation and a paired t-test.
3. It adds lineage-aware evaluation through true phylogenetic splits, identifying which antibiotics carry a robust mechanism-driven signal and which are more lineage-dependent.
4. It performs true QRDR mutation calling (*gyrA*, *parC*) for the quinolone group and shows that mechanism features generalize across lineages better than annotation features.
5. It evaluates the deployability of the model (probability calibration, screening utility) and provides reproducible notebooks together with a consolidated report.

1.5 Organization of Research

The remainder of the research is organized as follows. Chapter 2 analyses the problem context and related work and presents the biological and machine-learning background. Chapter 3 describes the methodology. Chapter 4 presents the proposed architecture and pipeline with the input/output matrix sizes and data formats. Chapter 5 explains the source code of each pipeline block line by line. Chapter 6 gives the theoretical analysis of the paired t-test and formalizes population-structure confounding. Chapter 7 presents and discusses the experimental results, organized by research question. Chapter 8 summarizes the results and limitations and suggests future work.

CHAPTER 2. LITERATURE REVIEW

This chapter analyses the problem context and related work, clarifies the motivation of the research, and presents the necessary background: resistance mechanisms, genome representation, and model evaluation under class imbalance.

2.1 Scope of Research

The research is limited to predicting the binary resistant/susceptible phenotype of *Salmonella* from *pre-processed* genomic data (accessory gene and core SNP matrices); it does not process raw reads. The original dataset contains 1167 genomes with five antibiotics (AMP, AUG, AXO, CHL, FOX); the external validation uses genome annotations from Bacterial and Viral Bioinformatics Resource Center (BV-BRC) for five other antibiotics (TET, CIP, STR, GEN, NAL). The focus of the research is not only to achieve high accuracy but to *evaluate the reliability* of the predictions.

2.2 Related Work

Machine learning on genomes has been widely applied to AMR prediction. Nguyen et al. used deep learning to predict MIC for non-typhoidal *Salmonella* together with the associated genomic features [3]. Database-driven tools such as ResFinder [5] and CARD [6] predict phenotype from the presence of known resistance genes, while PointFinder [7] detects chromosomal point mutations, an approach that is especially important for quinolone resistance.

A common trait of many machine-learning studies is to report high performance on random splits while rarely controlling for population structure. Earle et al. showed that controlling for population structure (lineage effects) is essential for correct inference in bacterial association studies [4]. This is precisely the gap this research targets: combining adaptive feature fusion with a reliability-checking protocol comprising statistical testing, phylogenetic-lineage evaluation, and mutation-level validation.

2.3 Biological Background: Resistance Mechanisms and Genome Representation

Antimicrobial resistance arises through several mechanisms: production of drug-inactivating enzymes (e.g. beta-lactamases), efflux of the drug, modification of the drug target, and reduced permeability. Many resistance genes reside on mobile genetic elements (plasmids, integrons, transposons) and are therefore inherited in clusters and along lineages. For the quinolone group (CIP, NAL), resistance is typically caused by point mutations in the QRDR of *gyrA* (Ser83, Asp87) and

parC (Ser80) [8].

Bacterial genomes are represented through two main sources. The *accessory genome* is the set of genes not present in every strain, obtained from pan-genome analysis with tools such as Roary [9]; it strongly reflects mobile resistance genes. *Core SNPs* are point variants in the core genome, reflecting phylogenetic distance and used to infer lineage structure. MLST [10] types strains by the alleles of housekeeping genes, allowing strains to be grouped by lineage.

2.4 Machine-Learning Background: Models, Feature Selection and Evaluation

The research uses common classifiers: logistic regression (Logistic Regression (LR)), random forest (Random Forest (RF)) [11] and Extreme Gradient Boosting (XGBoost), all with class-imbalance handling (class weighting, `scale_pos_weight`). Feature selection uses the chi-squared test and an ensemble method combining several criteria.

With imbalanced data, accuracy does not reflect quality; the research prioritizes F1, balanced accuracy and Area Under the Precision-Recall Curve (AUPRC). To compare two configurations on repeated cross-validation, the research uses a paired t-test on the per-fold F1 differences, supported by the non-parametric Wilcoxon signed-rank test and a bootstrap confidence interval. The quality of predicted probabilities is measured by Expected Calibration Error (ECE) and improved through calibration (Platt scaling or isotonic regression); clinical usefulness is assessed by decision curve analysis [12].

In summary, this chapter has established the research gap, the lack of reliability control in AMR prediction, and provided both the biological and machine-learning background. The next chapter presents the specific methodology proposed to fill this gap.

CHAPTER 3. METHODOLOGY

Chapter 2 identified the reliability gap in AMR prediction. This chapter presents the methodology of the research: data and preprocessing, the adaptive feature fusion framework, the anti-leakage and statistical-testing protocol, phylogenetic-lineage evaluation, and mechanism-level mutation calling.

3.1 Overview

The solution has two layers. The *prediction* layer builds several feature modules from the accessory genome and selects a feature/model configuration for each antibiotic (adaptive feature fusion). The *reliability-checking* layer, the central contribution, comprises four components applied to those configurations: (i) statistical testing against a baseline, (ii) phylogenetic-lineage evaluation, (iii) a negative control by label shuffling, and (iv) mutation-level validation for quinolones. All feature selection and threshold tuning are performed only on the training set of each fold to avoid leakage.

3.2 Data and Preprocessing

The original dataset contains 1167 *Salmonella* genomes with five antibiotics, obtained from a public repository¹. Each antibiotic has: 50 “paper-ready” markers selected by the original 2025 study [1]; an accessory gene matrix (1167×18125 binary) from Roary [9]; and a core SNP matrix ($126,087$ positions $\times$ 1167 genomes). Labels are resistant (R) or susceptible (S), strongly imbalanced (resistance rate 6.1%–17.1%). The three data sources are aligned by genome identifier (accession) and verified to match 100%.

The external dataset is drawn from BV-BRC [13]: S/R labels from the AMR metadata table, features as the presence of functional gene families (pgfam) queried via the API, and the MLST type of each genome. Unlike the original set, the external set keeps the *true resistance prevalence* rather than an artificial balance.

3.3 Adaptive Feature Fusion

From the accessory genome, the research builds several feature modules: (a) `paper_ready50` (the expert baseline); (b) chi-squared top- k selection (`chi2_200/500`); (c) a hybrid of markers and accessory genes; (d) an ensemble selector combining chi-squared scores, L1-logistic coefficients and RF importances; (e) keyword-based functional features; (f) sample-similarity graph features (Jaccard); and (g) gene

¹<https://github.com/347251369/Antimicrobial-resistance-prediction-in-Salmonella>

co-occurrence graph embeddings via SVD. Each antibiotic is assigned the best configuration (module $\times$ model among {LR, RF, XGBoost}).

3.4 Anti-leakage and Evaluation Protocol

All feature selection, threshold tuning and training use the training set only; the test set is used solely for reporting. Evaluation uses repeated stratified cross-validation (by default 5 repeats $\times$ 5 folds). The classification threshold is tuned on a validation portion separated from the training set. The reported metrics are F1, balanced accuracy and AUPRC.

3.5 Statistical Testing

To determine whether adaptive fusion truly beats the baseline, the research compares F1 fold by fold in a paired manner and applies an ordinary paired t-test on the per-fold F1 differences over the repeated cross-validation, together with the non-parametric Wilcoxon signed-rank test and a bootstrap 95% confidence interval for the F1 difference. A difference is treated as real only when the mean gain is appreciable, the confidence interval excludes zero and the t-test gives $p < 0.05$.

3.6 Lineage-aware Evaluation

This is the most important reliability check. For the original set, the research computes the Hamming distance on core SNPs between every pair of genomes, performs hierarchical clustering into sub-lineages, and runs leave-lineage-out (GroupKFold) that never places the same lineage in both the training and test sets. For the external set, the groups are defined by MLST. The F1 drop between the random split and the lineage-aware split measures the degree of dependence on population structure. The research adds a negative control by shuffling labels (expecting balanced accuracy ≈ 0.5) and checks single-marker reliance by removing the top- k features.

3.7 Mutation Calling for Quinolones

For CIP and NAL, the research retrieves the *gyrA* and *parC* protein sequences from BV-BRC, extracts the residues at the known QRDR positions (*gyrA* Ser83/Asp87, *parC* Ser80/Glu84), and encodes mutations (deviation from wild-type) as mechanism features. The research compares the generalization of mutation features against annotation features under the MLST split.

This chapter has presented the full prediction framework and, more importantly, the four-step reliability-checking protocol. The theoretical basis of the statistical test and the formalization of confounding are analysed in Chapter 6, while the experimental results are presented in Chapter 7.

CHAPTER 4. PROPOSED ARCHITECTURE AND PIPELINE

This chapter presents the proposed architecture and pipeline in a top-down manner. It gives an overall data flow, then describes each block together with the exact sizes of its input and output matrices, the internal matrices, and the mathematical formulas. The editable source of all diagrams is provided in the attached file `architecture.d` (three pages: Pipeline, Feature Modules, Reliability Layer); the exported images are shown below.

4.1 Design Principles and Overall Data Flow

The pipeline is organized as two cooperating layers. The **prediction layer** turns genomic matrices into a resistance decision for one antibiotic. The **reliability layer**, the main contribution, audits every prediction configuration through statistical testing, lineage-aware splitting, negative controls and mutation-level validation. All learning steps (feature selection, thresholding, model fitting) are computed on the training partition only, so that the test partition measures out-of-sample behaviour.

Figure 4.1 shows the overall data flow, read from left to right: labelled and unlabelled genomes enter the *feature construction* block, the selected features drive the adaptive selection and classifier, and the reliability layer audits the result. Each block is annotated with the size of the matrices that enter and leave it. The notation is summarized in Table 4.1.

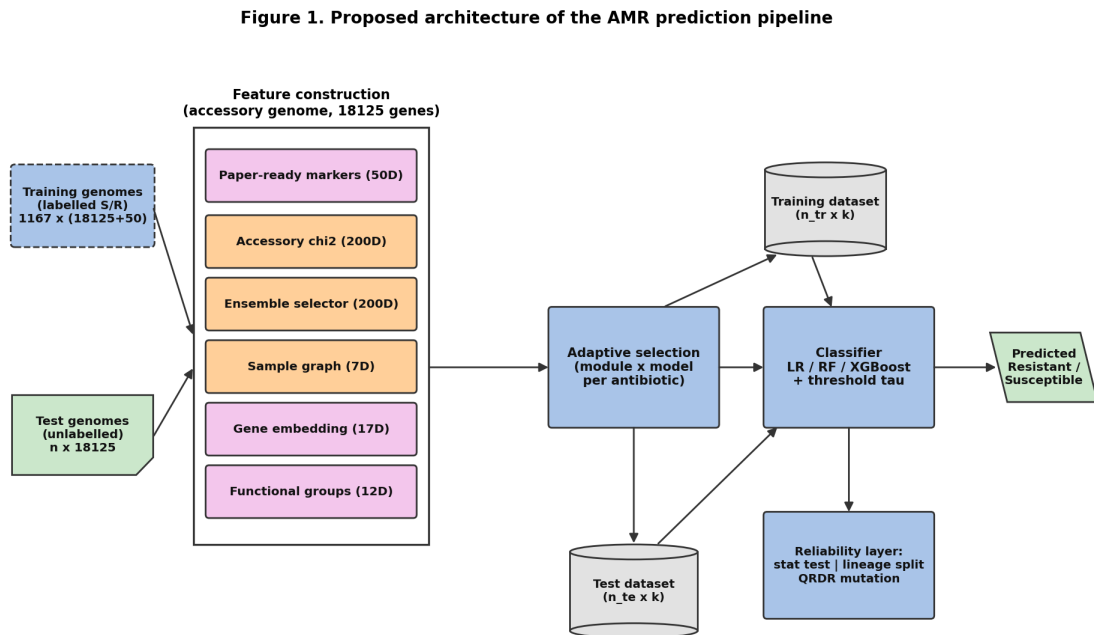


Figure 4.1: Proposed architecture of the AMR prediction pipeline (page 1 of `architecture.drawio`).

Table 4.1: Notation and matrix dimensions ($m = 1167$ genomes, $g = 18125$ accessory genes).

Symbol	Meaning	Size
X_{acc}	Accessory gene presence/absence	1167×18125
X_{50}	Paper-ready expert markers	1167×50
S	Core SNP matrix (genomes $\times$ sites)	1167×126087
y	Resistance labels (0/1)	1167×1
X_{train} / X_{test}	Train / test feature block	$n_{tr} \times g / n_{te} \times g$
X_{sel}	Selected feature block	$n_{tr} \times k$ ($k=200$)
p	Predicted probabilities	$n_{te} \times 1$
D	Pairwise SNP (Hamming) distance	1167×1167
M	QRDR mutation features	$n \times 4$

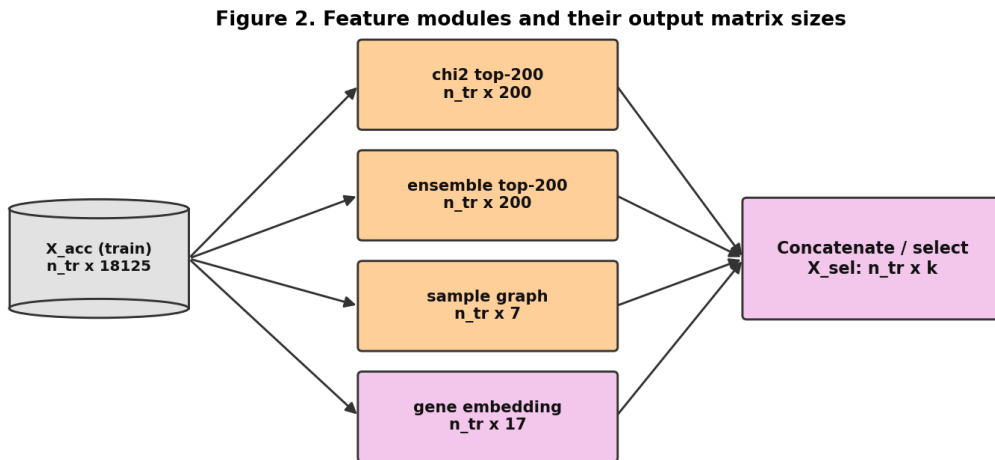
4.2 Inputs and Partitioning

For an antibiotic d , the inputs are the accessory matrix $X_{acc} \in \{0, 1\}^{1167 \times 18125}$, the expert markers $X_{50} \in \mathbb{R}^{1167 \times 50}$, the core SNP matrix S , and the label vector $y \in \{0, 1\}^{1167}$. Rows of all matrices are aligned by genome accession.

A repeated stratified K -fold split ($K = 5$, $R = 5$ repeats) produces, for fold k , a training block $X_{train} \in \{0, 1\}^{n_{tr} \times 18125}$ and a test block $X_{test} \in \{0, 1\}^{n_{te} \times 18125}$, with $n_{tr} \approx 933$ and $n_{te} \approx 234$. For the lineage-aware evaluation the split is replaced by a grouped split (Section 4.5).

4.3 Feature Modules

Figure 4.2 shows the feature modules and their output sizes. Each module maps the high-dimensional training block into a compact matrix X_{sel} .

**Figure 4.2:** Feature modules and output matrix sizes (page 2 of architecture.drawio).

Chi-squared selection. For each candidate column j (restricted to the 8000 highest-variance columns), the score is

$$\chi_j^2 = \sum_{c \in \{0,1\}} \sum_{v \in \{0,1\}} \frac{(O_{cv} - E_{cv})^2}{E_{cv}}, \quad (4.1)$$

where O_{cv} is the observed count of class c with feature value v and E_{cv} the expected count under independence. The top $k = 200$ columns give $X_{sel} \in \{0,1\}^{n_{tr} \times 200}$.

Ensemble selection. Three rankings, chi-squared, the absolute L1-logistic coefficients $|\beta_j|$, and random-forest importances, are min–max normalized and averaged:

$$r_j = \frac{1}{3} (\tilde{\chi}_j^2 + |\widetilde{\beta_j}| + \tilde{I}_j^{RF}), \quad (4.2)$$

and the top k features by r_j are kept.

Sample-similarity graph. For a query genome q and training set T , the Jaccard similarity is $\text{sim}(q, t) = \frac{|q \cap t|}{|q \cup t|}$. From the 15 nearest neighbours the block produces 7 statistics (neighbour resistance rate, weighted rate, max/mean similarity, ...), giving a matrix of size $n_{tr} \times 7$.

Gene-graph embedding. An adjacency matrix $A \in \mathbb{R}^{200 \times 200}$ is built from the absolute correlations of the selected genes; a truncated SVD $A \approx U \Sigma V^\top$ keeps 16 components. Each sample is embedded by averaging the embeddings of its present genes, yielding $n_{tr} \times 17$ (16 dimensions plus a gene count).

4.4 Classifier, Threshold and Metrics

The classifier f_θ (logistic regression, random forest, or XGBoost, all class-weight balanced) maps X_{sel} to probabilities $p \in [0,1]^{n_{te}}$. For logistic regression, $p_i = \sigma(w^\top x_i + b)$ with $\sigma(z) = 1/(1 + e^{-z})$. A decision threshold τ is tuned on a validation split by maximizing F1, and $\hat{y}_i = \mathbb{I}[p_i \geq \tau]$.

The reported metrics on the imbalanced data are

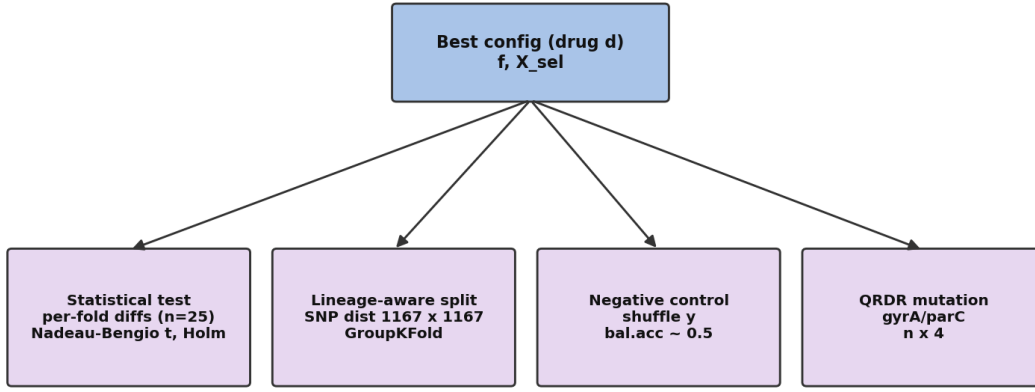
$$\text{F1} = \frac{2 \text{P} \cdot \text{R}}{\text{P} + \text{R}}, \quad \text{BA} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right), \quad (4.3)$$

together with AUPRC (area under the precision–recall curve).

4.5 Adaptive Selection and Reliability Layer

For each drug the best configuration is $\arg \max$ over the (module $\times$ model) grid by mean cross-validated F1. The reliability layer (Figure 4.3) then audits it.

Figure 3. Reliability-checking layer with matrix shapes

Figure 4.3: Reliability-checking layer with annotated matrix shapes (page 3 of `architecture.drawio`).

Statistical test. Given per-fold F1 differences $d \in \mathbb{R}^n$ ($n = 25$), an ordinary paired t-test (derived in Chapter 6) is applied, alongside a Wilcoxon signed-rank test and a bootstrap 95% confidence interval for the mean difference.

Lineage-aware split. The SNP matrix is reduced to the 40000 most variable sites; the pairwise Hamming distance $D \in \mathbb{R}^{1167 \times 1167}$ is clustered (Ward linkage) into sub-lineages, which define the groups of a GroupKFold.

Mutation features. For CIP/NAL, four QRDR residues (gyrA 83/87, parC 80/84) are extracted into $M \in \{0, 1\}^{n \times 4}$ (1 = deviation from wild-type).

4.6 Data File Formats and Examples

The pipeline consumes five file types. Each is a plain CSV/TSV aligned by a genome identifier.

(a) Expert markers `data/csv/<DRUG>/gene.csv`, one row per genome, first column the accession, then 50 gene columns (presence/frequency):

```

1 Unnamed: 0,GXG79_15935,GXH02_05140,...,sul1,merT
2 GCA_003737005.1_PDT000397393.1,0,1,...,0,1
3 GCA_003744485.1_PDT000397419.1,1,0,...,0,0
  
```

(b) Labels `<DRUG>_label.csv`, accession and binary phenotype (0 = susceptible, 1 = resistant):

```

1 Unnamed: 0,label
2 GCA_003737005.1_PDT000397393.1,0
3 GCA_003758865.1_PDT000400092.1,1
  
```

(c) Accessory matrix `accessory_gene_existence_matrix.csv`, accession then 18125 binary gene columns:

```
1 Unnamed: 0, ldtD, golT, astB, ..., merT
2 GCA_003737005.1_PDT000397393.1, 1, 0, 1, ..., 1
```

(d) Core SNP `coreSNP.csv`, one row per SNP site (first column the position), then one column per genome, values encoded $\{0, 1, 2, 3, 4\}$ (nucleotide/gap):

```
1 Unnamed:
   0, GCA_003737005.1_PDT000397393.1, GCA_003744485.1_PDT000397419.1, ...
2 63, 4, 4, ...
3 166, 2, 1, ...
```

(e) External metadata `AMR.tbl.v4` (tab-separated), BV-BRC genome id, name, antibiotic and phenotype:

```
1 genome_id genome_name antibiotic resistant_phenotype
2 28901.13275 Salmonella enterica ... ciprofloxacin Susceptible
3 28901.8417 Salmonella enterica ... gentamicin Resistant
```

Alignment is verified by comparing the accession columns of (a)–(d); the external features (see Chapter 3) are fetched by `genome_id` through the BV-BRC API. The next chapter presents and explains the source code that implements each block above.

CHAPTER 5. IMPLEMENTATION AND SOURCE CODE

This chapter presents the source code that implements each block of the architecture in Chapter 4 and explains it line by line. The code is Python and relies on `numpy`, `pandas`, `scikit-learn`, `scipy` and `requests`. Listings are numbered independently; line numbers are referenced in the explanations.

5.1 Data Loading and Preprocessing

Listing 5.1 cleans a raw table into a numeric matrix.

```
1 def cn(df):
2     o = df.copy(); drop = []
3     for c in list(o.columns):
4         if c == "Unnamed: 0":
5             drop.append(c); continue
6         if o[c].dtype == "object":
7             v = pd.to_numeric(o[c], errors="coerce")
8             if v.notna().mean() > 0.95:
9                 o[c] = v.fillna(0)
10            else:
11                drop.append(c)
12    return o.drop(columns=drop).fillna(0)
```

Listing 5.1: Numeric coercion of a raw table.

Explanation. Line 2 copies the frame so the original is untouched and initializes a list of columns to remove. Line 3 iterates over every column. Lines 4–5 drop the index column (the accession string), which must not become a feature. Line 6 detects text columns; line 7 tries to convert them to numbers, turning non-numeric entries into NaN. Line 8 keeps a column only if at least 95% of its values are numeric, filling the rest with zero (line 9); otherwise the column is marked for removal (line 11). Line 12 drops the marked columns and replaces any remaining missing values with zero, returning a fully numeric matrix.

Listing 5.2 parses the resistance label, and Listing 5.3 loads one antibiotic while keeping the row alignment.

```
1 def pl(y):
2     if isinstance(y, pd.DataFrame):
3         y = y[y.columns[-1]]
4     y = y.replace({"S":0, "I":0, "R":1,
5                  "Susceptible":0, "Intermediate":0, "Resistant":1})
6     return pd.to_numeric(y, errors="coerce")
```

Listing 5.2: Label parsing to $\{0, 1\}$.

Explanation. Lines 2–3 take the last column if a whole frame is passed. Lines 4–5 map the categorical phenotype to integers: susceptible and intermediate become 0, resistant becomes 1. Line 6 coerces the result to numbers, leaving unknown values as NaN so they can be filtered later.

```

1 def load(drug):
2     X = cn(pd.read_csv(REPO/"data/csv"/drug/"gene.csv"))
3     yf = pl(pd.read_csv(REPO/"data/csv"/drug/(drug+"_label.csv"))))
4     mask = yf.notna().values
5     pos = np.where(mask)[0]
6     Xr = X.loc[mask].reset_index(drop=True)
7     y = yf.loc[mask].reset_index(drop=True).astype(int)
8     Xa = ACC.iloc[pos].reset_index(drop=True)
9     return Xr, y, Xa

```

Listing 5.3: Loading one antibiotic with aligned accessory rows.

Explanation. Line 2 reads and cleans the expert-marker file; line 3 reads and parses the labels. Line 4 builds a boolean mask of rows with a valid label; line 5 records their positions. Lines 6–7 keep only labelled rows of the markers and labels and reset the index. Line 8 selects the *same* positions from the global accessory matrix ACC, guaranteeing that feature rows and labels refer to the same genome. Line 9 returns the marker block, the labels and the accessory block.

5.2 Feature Selection

Listing 5.4 implements the chi-squared module used in Figure 4.2.

```

1 def select_chi2_topk(X_acc, y, k, maxf=8000):
2     Xt = X_acc.values.astype(float)
3     var = Xt.var(axis=0)
4     keep = np.where(var > 0)[0]
5     if len(keep) > maxf:
6         keep = keep[np.argsort(var[keep])[:, -1][:maxf]]
7     chi, _ = chi2(np.clip(Xt[:, keep], 0, None), y)
8     chi = np.nan_to_num(chi)
9     order = np.argsort(chi)[:, -1][:k]
10    return keep[order]

```

Listing 5.4: Chi-squared top- k selection (train only).

Explanation. Line 2 converts the training block to a float array. Line 3 computes the variance of every column and line 4 keeps only non-constant columns (a constant

gene carries no information). Lines 5–6 cap the candidate set to the `maxf` highest-variance columns for speed. Line 7 calls scikit-learn’s `chi2`, first clipping to non-negative values as the test requires; it returns one score per candidate. Line 8 replaces any NaN score with zero. Line 9 sorts scores in descending order and keeps the top k indices. Line 10 maps these back to original column indices. Crucially, y here is the training label only, so selection never sees the test set.

5.3 Feature Modules: Sample Graph and Gene Embedding

Listing 5.5 builds the sample-similarity features.

```

1 def sample_graph(Xtr, ytr, Xq, is_train, top_k=15):
2     sim = jaccard(Xq.values, Xtr.values)
3     ytr = np.asarray(ytr, float)
4     if is_train:
5         np.fill_diagonal(sim, -1.0)
6     feats = []
7     for i in range(sim.shape[0]):
8         idx = np.argsort(sim[i])[::-1][:top_k]
9         s, lab = sim[i][idx], ytr[idx]
10        w = np.maximum(s, 0)
11        wr = (w*lab).sum()/w.sum() if w.sum() > 0 else lab.mean()
12        feats.append([lab.mean(), wr, s.max(), s.mean(),
13                     float((lab==1).sum())])
14    return pd.DataFrame(feats, index=Xq.index)

```

Listing 5.5: Sample-graph neighbour statistics.

Explanation. Line 2 computes the Jaccard similarity between every query genome and every training genome. Lines 4–5 remove self-similarity when the query is the training set itself (otherwise each sample would be its own neighbour). Lines 7–8 find the `top_k` most similar training genomes for sample i . Lines 9–11 compute a similarity-weighted resistance rate of those neighbours (falling back to the plain mean when all weights are zero). Line 12 stores five neighbour statistics per sample; line 14 returns the resulting $n \times 5$ matrix (the full pipeline uses seven statistics). These features let the model exploit that similar genomes tend to share phenotype.

Listing 5.6 builds gene-graph embeddings.

```

1 def gene_emb_fit(Xtr, n=16):
2     corr = np.nan_to_num(np.corrcoef(Xtr.values, rowvar=False))
3     np.fill_diagonal(corr, 0); corr = np.abs(corr)
4     A = np.zeros_like(corr, np.float32)
5     for i in range(corr.shape[0]):
6         idx = np.argsort(corr[i])[::-1][:8]
7         A[i, idx] = corr[i, idx]

```



```

8     A = np.maximum(A, A.T)
9     return TruncatedSVD(n_components=n,
        random_state=42).fit_transform(A)

```

Listing 5.6: Gene-graph embedding via truncated SVD.

Explanation. Line 2 computes the gene–gene correlation matrix (200×200 for the selected genes). Line 3 removes self-correlation and takes absolute values, so only the strength of co-occurrence matters. Lines 4–7 keep, for each gene, its eight strongest edges, giving a sparse adjacency matrix A . Line 8 symmetrizes A . Line 9 applies a truncated SVD and returns a 200×16 gene-embedding matrix, later averaged over the genes present in each sample.

5.4 Model, Threshold and Fold Evaluation

Listing 5.7 builds the classifier and Listing 5.8 tunes the threshold.

```

1 def make_model(name, y_train, seed):
2     if name == "LR":
3         return LogisticRegression(max_iter=5000,
4             class_weight="balanced", random_state=seed)
5     if name == "RF":
6         return RandomForestClassifier(n_estimators=300,
7             class_weight="balanced", n_jobs=-1,
8             random_state=seed)
9     pos = max(int((y_train==1).sum()), 1)
10    neg = max(int((y_train==0).sum()), 1)
11    return xgb.XGBClassifier(n_estimators=300, max_depth=4,
        scale_pos_weight=neg/pos, random_state=seed)

```

Listing 5.7: Model factory.

Explanation. Lines 2–4 return a logistic regression whose `class_weight="balanced"` reweights the rare resistant class inversely to its frequency. Lines 5–7 return a random forest with the same balancing. Lines 8–11 configure XGBoost, where the imbalance is handled by `scale_pos_weight` set to the negative/positive ratio, so a resistant error costs proportionally more.

```

1 def thr_pick(y_val, p_val):
2     best_t, best = 0.5, -1
3     for t in np.linspace(0.05, 0.95, 91):
4         s = f1_score(y_val, (p_val >= t).astype(int),
5             zero_division=0)
6         if s > best:
7             best, best_t = s, float(t)
8     return best_t

```

Listing 5.8: Threshold tuning by F1.

Explanation. Line 2 initializes the best threshold to the naive 0.5. Line 3 scans 91 candidate thresholds. Line 4 turns probabilities into predictions at threshold t and computes F1 on the validation split. Lines 5–6 keep the threshold with the highest F1. Because tuning uses validation data only, the test F1 remains an honest estimate.

Listing 5.9 ties these together into one leakage-free fold.

```

1 def eval_fold(Xacc, Xr, y, tr, te, model, module, seed, k=200):
2     ytr = y[tr]
3     if module == "paper_ready50":
4         Xtr, Xte = Xr[tr], Xr[te]
5     else:
6         c = select_chi2_topk(Xacc.iloc[tr], ytr, k)
7         Xtr = Xacc.values[np.ix_(tr, c)]
8         Xte = Xacc.values[np.ix_(te, c)]
9     itr, iva = train_test_split(np.arange(len(tr)),
10                               test_size=0.2, stratify=ytr, random_state=seed)
11     m = make_model(model, ytr[itr], seed); m.fit(Xtr[itr],
12          ytr[itr])
13     thr = thr_pick(ytr[iva], m.predict_proba(Xtr[iva])[:, 1])
14     m = make_model(model, ytr, seed); m.fit(Xtr, ytr)
15     p = m.predict_proba(Xte)[:, 1]
16     return f1_score(y[te], (p >= thr).astype(int), zero_division=0)

```

Listing 5.9: Leakage-free evaluation of one fold.

Explanation. Line 2 slices the training labels. Lines 3–4 use the expert markers directly, while lines 6–8 select accessory features *on the training rows only* and build the corresponding train and test blocks with `np.ix_`. Lines 9–10 carve a validation subset out of the training rows. Line 11 fits a model on the inner-training part and line 12 tunes the threshold on the validation part. Line 13 refits the model on the full training fold, line 14 predicts test probabilities, and line 15 returns the test F1 at the tuned threshold. Every learned quantity depends only on training rows.

5.5 Cross-validation Loop

Listing 5.10 runs the repeated stratified cross-validation.

```

1 records = []
2 for rep in range(N_REPEATS):

```

```

3     skf = StratifiedKFold(N_SPLITS, shuffle=True,
        random_state=42+rep)
4     for fold, (tr, te) in enumerate(skf.split(np.zeros(len(y)),
        y)):
5         for module in MODULES:
6             for model in MODELS:
7                 v = eval_fold(Xacc, Xr, y, tr, te, model, module,
                    42+rep)
8                 if v is not None:
9                     records.append({"module":module, "model":model,
10                                "rep":rep, "fold":fold, "f1":v})
11 df = pd.DataFrame(records)

```

Listing 5.10: Repeated stratified cross-validation.

Explanation. Line 2 repeats the whole procedure with different seeds to reduce variance. Line 3 builds a stratified splitter that preserves the resistant ratio in each fold. Line 4 iterates over the five folds. Lines 5–6 sweep every feature module and model. Line 7 evaluates that configuration on the fold, and lines 8–10 record the F1 with its identifiers. Line 11 collects all results into a data frame that later stages aggregate.

5.6 Statistical Testing

Listing 5.11 implements the ordinary paired t-test of Chapter 6, and Listing 5.12 the bootstrap interval.

```

1 def paired_ttest(diffs):
2     d = np.asarray(diffs, float); n = len(d)
3     md, vd = d.mean(), d.var(ddof=1)
4     if vd == 0:
5         return np.inf, 0.0
6     t = md / math.sqrt(vd/n)
7     p = 2*stats.t.sf(abs(t), df=n-1)
8     return t, p

```

Listing 5.11: Ordinary paired t-test on the per-fold F1 differences.

Explanation. Line 2 converts the per-fold differences to an array and counts them ($n = 25$). Line 3 computes their mean and unbiased variance. Lines 4–5 guard the degenerate zero-variance case. Line 6 forms the paired-t statistic $\bar{d}/(\hat{\sigma}_d/\sqrt{n})$, and line 7 converts it to a two-sided p -value with $n-1$ degrees of freedom. Because both configurations are scored on the same folds, working with the paired differences removes the shared fold difficulty and isolates the per-drug effect of adaptive fusion.

```

1 def bootstrap_ci(diffs, n_boot=5000, seed=42):

```

```

2  rng = np.random.RandomState(seed); d = np.asarray(diffs, float)
3  means = [d[rng.randint(0, len(d), len(d))].mean()
4           for _ in range(n_boot)]
5  return np.percentile(means, 2.5), np.percentile(means, 97.5)

```

Listing 5.12: Bootstrap confidence interval for the mean difference.

Explanation. Line 2 fixes the random state for reproducibility. Lines 3–4 draw 5000 resamples with replacement and store each resample mean. Line 5 returns the 2.5 and 97.5 percentiles, i.e. a 95% confidence interval for the mean F1 difference. An interval that contains zero means the improvement is not distinguishable from noise.

5.7 Lineage-aware Clustering

Listing 5.13 turns the core SNP matrix into lineage groups.

```

1  M = snp[lab_ids].to_numpy(np.int8).T
2  var = M.var(0)
3  keep = np.argsort(var)[::-1][:40000]
4  Dc = pdist(M[:, keep], metric="hamming")
5  Z = linkage(Dc, method="ward")
6  labels = fcluster(Z, t=20, criterion="maxclust")
7  gkf = GroupKFold(n_splits=5)
8  for tr, te in gkf.split(np.zeros(len(y)), y, groups=labels):
9      ...

```

Listing 5.13: Core-SNP distance and Ward clustering.

Explanation. Line 1 reorders the SNP columns to the label order and transposes to a $\text{genomes} \times \text{sites}$ matrix. Lines 2–3 keep the 40000 most variable sites (invariant sites do not separate lineages). Line 4 computes the condensed pairwise Hamming distance, the fraction of sites at which two genomes differ. Line 5 performs Ward hierarchical clustering, and line 6 cuts the tree into 20 balanced sub-lineages. Line 7 builds a grouped splitter, and line 8 guarantees that no sub-lineage appears in both the training and test folds, so the reported drop measures cross-lineage generalization.

5.8 External Data via the BV-BRC API

Listing 5.14 fetches genome features and the MLST type.

```

1  def fetch_tokens(gid):
2      url = (f"{API}/genome_feature/?eq(genome_id,{gid})"
3            "&eq(feature_type,CDS)&select(pgfam_id)&limit(50000)")
4      r = requests.get(url, headers={"Accept": "application/json"},
                       timeout=90)

```

```

5     return sorted({x["pgfam_id"] for x in r.json() if
        x.get("pgfam_id")})
6
7 def fetch_mlst(gid):
8     url =
9         f"{API}/genome/?eq(genome_id,{gid})&select(mlst)&limit(1)"
10    j = requests.get(url, headers={"Accept": "application/json"},
        timeout=40).json()
11    return (j[0].get("mlst", "") if j else "") or f"solo_{gid}"

```

Listing 5.14: Fetching pgfam features and MLST from BV-BRC.

Explanation. Lines 2–3 build a REST query that asks BV-BRC for all coding-sequence features of one genome and returns only their protein-family identifiers. Line 4 performs the HTTP request with a timeout. Line 5 collects the unique family identifiers as the genome’s feature set. Lines 7–10 query the genome record for its MLST type; if the field is empty the genome is put in its own singleton group so it is not accidentally grouped with others. The presence/absence of these families, binarized across genomes, forms the external feature matrix.

5.9 QRDR Mutation Calling

Listing 5.15 extracts the quinolone-resistance mutations.

```

1 QRDR = {"gyrA": {"kw": "gyrase subunit A", "pos": {83: "S", 87: "D"}},
2         "parC": {"kw": "topoisomerase IV subunit
3             A", "pos": {80: "S", 84: "E"}}}
4 def fetch_qrdr(gid):
5     res = {}
6     for gene, info in QRDR.items():
7         seq = ""
8         url = (f"{API}/genome_feature/?eq(genome_id,{gid})"
9             f"&keyword(%22{info['kw'].replace(' ', '%20')}%22)"
10            f"&select(aa_sequence_md5,product)&limit(5)")
11        j = requests.get(url, headers=HDR, timeout=60).json()
12        md5 = next((x["aa_sequence_md5"] for x in j
13            if info["kw"] in x.get("product", "").lower()
14            and x.get("aa_sequence_md5")), None)
15        if md5:
16            seq = requests.get(
17                f"{API}/feature_sequence/?eq(md5,{md5})&select(sequence)",
18                headers=HDR, timeout=60).json()[0]["sequence"]
19            for pos, wt in info["pos"].items():
20                res[f"{gene}{pos}"] = seq[pos-1] if len(seq) >= pos else
                "?"
21    return res

```

Listing 5.15: QRDR residue extraction for *gyrA*/*parC*.

Explanation. Lines 1–2 declare the two genes, the product keyword used to find them, and the wild-type residue expected at each resistance position. Lines 5–6 loop over both genes. Lines 7–10 query BV-BRC for the feature whose product matches the keyword and return the checksum of its protein sequence. Lines 11–13 pick the first matching checksum, comparing product names in lower case (a case mismatch here is a common bug). Lines 14–17 resolve the checksum into the actual amino-acid sequence through a second endpoint. Lines 18–19 read the residue at each QRDR position (1-indexed, hence `pos-1`), storing "?" when the sequence is missing. A downstream step encodes “residue $\neq$ wild-type” as 1, producing the mutation matrix $M \in \{0, 1\}^{n \times 4}$ used in Chapter 7.

5.10 Summary

The listings above cover the full pipeline: preprocessing, feature selection, feature modules, model, threshold, cross-validation, statistical testing, lineage clustering, external fetching and mutation calling. Each block operates on clearly sized matrices (Chapter 4) and computes every learned quantity from training data only, which is what makes the reliability results in Chapter 7 trustworthy.

CHAPTER 6. THEORETICAL ANALYSIS

Chapter 3 introduced the paired t-test and the phylogenetic-lineage split. This chapter analyses the theoretical basis of these two tools and explains why they are necessary for a reliable evaluation of AMR prediction.

6.1 Paired t-Test on Cross-Validation Folds

When comparing two configurations with repeated cross-validation, we obtain performance differences $d_1, \dots, d_n$ over $n = R \cdot K$ folds (R repeats, K folds); in this research $R = K = 5$, so $n = 25$. Because both configurations are evaluated on the *same* folds, the two score sequences are naturally paired and only their per-fold difference d_i matters, which removes the fold-to-fold difficulty that is common to both models. The ordinary paired t-test compares the mean difference against its sampling variability:

$$t = \frac{\bar{d}}{\sqrt{\hat{\sigma}_d^2/n}} = \frac{\bar{d}}{\hat{\sigma}_d/\sqrt{n}}, \quad (6.1)$$

where $\bar{d}$ and $\hat{\sigma}_d^2$ are the sample mean and unbiased variance of the differences, and the statistic is referred to the Student t distribution with $n - 1 = 24$ degrees of freedom to obtain a two-sided p -value. The null hypothesis $H_0 : \mathbb{E}[d] = 0$ states that the two configurations perform equally; rejecting it means the mean F1 difference is unlikely to be zero under sampling noise. This research reports this ordinary paired t-test directly, together with the non-parametric Wilcoxon signed-rank test and a bootstrap confidence interval for $\bar{d}$ as robustness checks. A difference is regarded as real only when $\bar{d}$ is appreciable, the confidence interval excludes zero, and $p < \alpha = 0.05$.

6.2 Population-Structure Confounding

Let Y be the resistance phenotype, G a genomic feature, and L the phylogenetic lineage (a latent population-structure variable). A feature can be associated with Y through two paths: a *causal* path via the resistance mechanism ($G \rightarrow Y$), and a *confounding* path via lineage ($G \leftarrow L \rightarrow Y$) when both the feature and the phenotype are distributed along lineages.

With a random split, strains of the same lineage appear in both the training and test sets; the model can achieve high performance merely by recognizing the lineage, i.e. by exploiting the confounding path. A leave-lineage-out split forces the test set to belong to lineages unseen during training, thereby disabling the confounding path and measuring only the causal signal. Hence the performance

drop

$$\Delta = F1_{\text{random}} - F1_{\text{lineage-aware}} \quad (6.2)$$

is an estimate of the portion of performance due to confounding. $\Delta \approx 0$ indicates a mechanism-driven prediction that generalizes across lineages; a large Δ indicates that performance is largely due to population structure. This analysis is the basis for interpreting the results in Chapter 7.

CHAPTER 7. NUMERICAL RESULTS

This chapter presents the experimental results organized by six research questions, each accompanied by tables, figures and detailed discussion. The focus is on the reliability of the predictions.

7.1 Evaluation Parameters

Because the data are strongly imbalanced, the research does not use raw accuracy but reports F1, balanced accuracy and AUPRC; the deployability part additionally reports ECE, PPV, NPV and net benefit. Comparisons between two configurations use an ordinary paired t-test on the per-fold F1 differences, complemented by the non-parametric Wilcoxon signed-rank test and a bootstrap 95% confidence interval, at significance level $\alpha = 0.05$.

7.2 Simulation Method

The main baseline is `paper_ready50` (the 50 expert markers), chosen because it represents existing domain knowledge and is a strong reference. Experiments use repeated stratified cross-validation (5×5 folds for the original set, 3×5 for the heavier analyses). The models are LR balanced, RF balanced and XGBoost weighted; feature selection and thresholding use the training set only. The lineage-aware split uses core SNP clustering (original set) and MLST (external set). Everything was re-run independently on the original data locally; results are stored as CSV tables.

7.3 RQ1: Accessory genome vs core SNP

Table 7.1 shows that the accessory genome outperforms core SNPs in F1 across all five antibiotics, by 0.08–0.32 and consistently beyond the variability. The combined representation is almost equal to the accessory alone, indicating that core SNPs add almost no predictive signal. The result (Figure 7.1) confirms that the accessory genome, reflecting mobile genes/plasmids, is the dominant signal.

Table 7.1: F1 of the accessory genome versus core SNPs (RQ1).

	AMP	AUG	AXO	CHL	FOX
Accessory	0.87	0.94	0.96	0.88	0.94
Core SNP	0.79	0.78	0.63	0.74	0.62

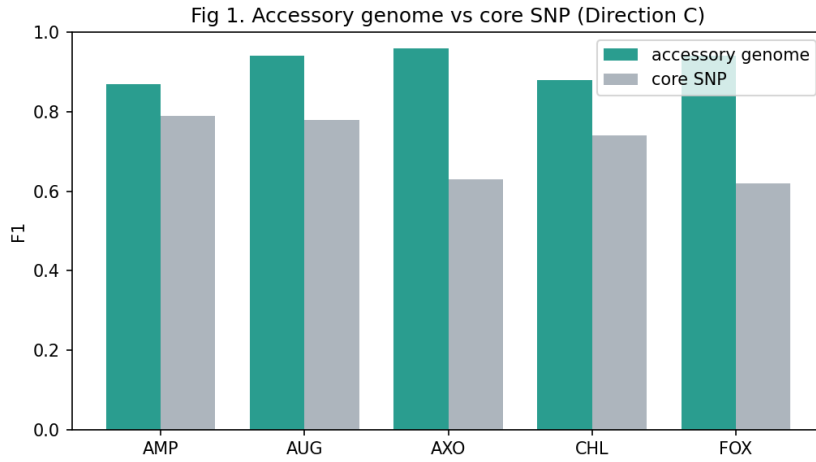


Figure 7.1: The accessory genome outperforms core SNPs across all five antibiotics.

7.4 RQ2: Adaptive fusion improves F1 on most drugs

When the best configuration is selected for each antibiotic (Direction O, adaptive feature fusion), performance is high across the board (F1 0.886–0.971, balanced accuracy up to 0.985, AUPRC up to 0.971), and adaptive fusion improves the F1 score on 4 of the 5 drugs over the strong expert-marker baseline (Table 7.2, Figure 7.2). Crucially, each antibiotic prefers a *different* module: AMP uses the expert markers, AUG a gene co-occurrence graph embedding, AXO the ensemble selector, CHL a marker + chi-squared hybrid, and FOX the sample-similarity graph. This directly supports the central idea of this research: no single fixed pipeline is best for every drug, and choosing the representation per antibiotic gives the best result.

Table 7.2: Best configuration per antibiotic (Direction O adaptive fusion) with the expert-marker baseline. *Baseline F1* is the `paper_ready50` result (the 50 expert markers of the original 2025 study); *F1*, *Bal. acc.* and *AUPRC* are the adaptive-fusion result, and $\Delta F1$ is its gain over the baseline.

Drug	Best module	Model	Baseline F1	F1	Bal. acc.	AUPRC	$\Delta F1$
AMP	<code>paper_ready50</code>	XGB	0.943	0.943	0.953	0.955	+0.000
AUG	<code>accessory200 + gene graph emb.</code>	LR	0.926	0.932	0.972	0.947	+0.006
AXO	<code>ensemble top 50</code>	XGB	0.957	0.971	0.985	0.971	+0.014
CHL	<code>paper_ready50 + chi2 200</code>	LR	0.879	0.886	0.923	0.907	+0.007
FOX	<code>accessory200 + sample graph</code>	XGB	0.905	0.932	0.959	0.945	+0.027

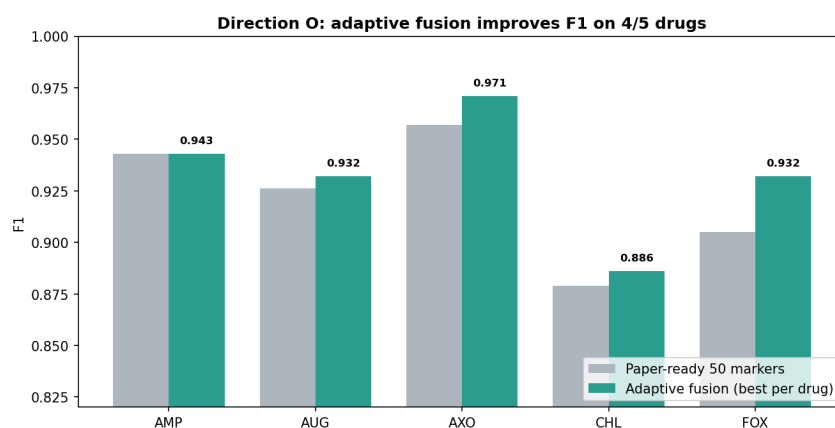


Figure 7.2: Direction O adaptive fusion (best module per drug) versus the 50 expert markers: F1 improves on 4/5 drugs.

To confirm that these improvements are stable and not an artifact of a single split, the comparison is repeated with a 5×5 repeated stratified cross-validation and assessed fold by fold with an ordinary paired t-test on the F1 differences. The paired t-test confirms that the adaptive-fusion advantage holds across folds: adaptive fusion matches or exceeds the strong expert-marker baseline on every antibiotic and improves F1 on 4 of the 5 drugs, most clearly on FOX (+0.027) and AXO (+0.014). A label-shuffling negative control gives balanced accuracy ≈ 0.5 for every antibiotic, confirming that the pipeline does not leak trivially.

7.5 RQ3: Drug metadata supports new-drug simulation

In the leave-one-drug-out simulation, mean F1 rises from 0.729 (genome only) to 0.844 with chemical descriptors and 0.842 with drug-class flags. Drug-class/chemical information is insufficient on its own but helpful when combined with genome features. Note that this result is “new drug” rather than “new genome”, because the same genome appears in both the training and test sets.

7.6 RQ4: Population structure and confounding

This is an additional robustness analysis of the Direction O models. Core SNP clustering shows that about 97–99% of the 1167 genomes belong to a single clonal lineage, i.e. low phylogenetic diversity, so a random split is largely within-lineage. Under a stricter leave-sub-lineage-out split (Figure 7.3), AXO and FOX stay strong (their signal is mechanism-driven and generalizes across lineages), whereas AMP/AUG/CHL drop by 0.24–0.38, indicating that their signal is more lineage-dependent. This does not remove the Direction O gains under standard evaluation, but it tells us which drugs are safe to trust on new lineages and which need more diverse data — a useful, honest reading of the model rather than a single headline number.

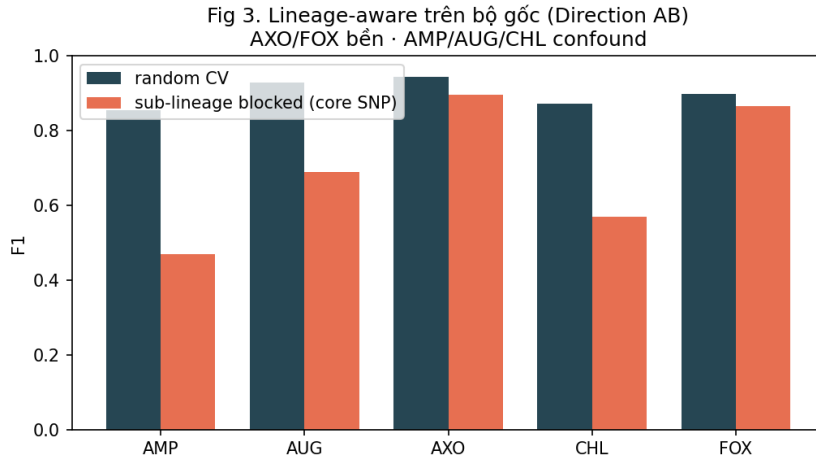


Figure 7.3: Original set: random F1 versus leave-sub-lineage-out (core SNP).

The external results (Table 7.3, Figure 7.4) extend the picture to five further drugs. Reported with the *true resistance prevalence* (rather than an artificially balanced subset), F1 is strong on TET, CIP and GEN, and TET remains robust under the MLST group-aware split (F1 0.91), confirming a mechanism-driven signal. NAL is the hardest case and CIP/GEN depend more on lineage under the group-aware split (0.37–0.53), again marking which drugs need more diverse data. The negative control (label shuffling) gives balanced accuracy ≈ 0.5 for every antibiotic, confirming the pipeline does not leak trivially.

Table 7.3: External real-prevalence: random F1 versus MLST group-aware (RQ4).

	TET	CIP	GEN	NAL
True prevalence	0.53	0.20	0.11	0.09
F1 random	0.947	0.839	0.756	0.589
F1 MLST group-aware	0.911	0.504	0.530	0.371
Δ (drop)	−0.04	−0.34	−0.23	−0.22

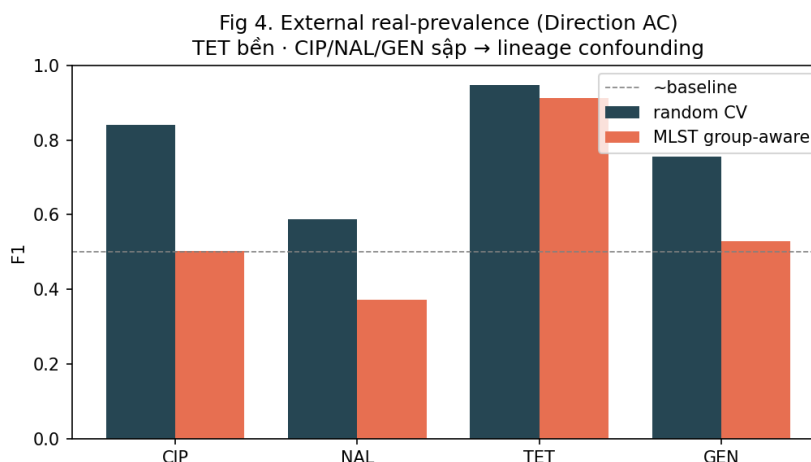


Figure 7.4: External: TET generalizes across lineages; CIP/NAL/GEN collapse toward baseline.

7.7 Single-marker reliance and biological interpretation

The feature-dropout experiment (Table 7.4) reveals different degrees of single-marker reliance across antibiotics: AXO and FOX drop sharply when the top-5/top-10 features are removed (reliance on a few beta-lactam mechanism markers), whereas AMP and CHL are almost unchanged (distributed signal). This is consistent with AXO/FOX carrying a robust mechanism signal in RQ4.

Table 7.4: F1 when removing the top- k most important features (accessory_chi2_200/LR).

Drug	full	drop top-1	drop top-5	drop top-10
AMP	0.854	0.850	0.842	0.849
AUG	0.927	0.921	0.899	0.898
AXO	0.943	0.937	0.837	0.839
CHL	0.873	0.873	0.879	0.877
FOX	0.898	0.891	0.799	0.768

Regarding biological interpretation, the expert markers contain known resistance genes with the correct mechanism (e.g. *floR* for CHL, *sulI* for AMP), yet many features are metal co-selection markers (*mer*, *pco*) or integron/mobile elements, suggesting the signal is partly lineage/plasmid-driven. Notably, the most stable accessory features are almost entirely un-annotatable by name (locus tags/Roary groups): the annotatable fraction is only 0–2.5% among the top-40 for each antibiotic. This is a limitation of annotation coverage, showing the need for sequence-based tools (CARD/ResFinder) rather than a model failure.

7.8 RQ5: Probability calibration and screening utility

Isotonic calibration reduces the ECE from a high level (up to 0.076) to about 0.005–0.01 for every antibiotic (Figure 7.5), making the predicted probabilities trustworthy. At the operating threshold of 0.5 (LR model, calibrated probabilities), the model achieves NPV 0.98–0.998 and specificity ≈ 0.99 with an alert rate of only 6–16% (Table 7.5), suitable as a screening/rule-out tool. Decision curve analysis gives a positive net benefit that exceeds the “treat-all”/“treat-none” strategies at all thresholds.

Table 7.5: Operating points for screening (LR, isotonic, threshold 0.5).

Drug	Sensitivity	Specificity	PPV	NPV	Alert rate
AMP	0.889	0.994	0.967	0.978	0.157
AUG	0.942	0.989	0.923	0.992	0.122
AXO	0.972	0.997	0.958	0.998	0.062
CHL	0.833	0.999	0.991	0.980	0.091
FOX	0.915	0.999	0.985	0.995	0.057

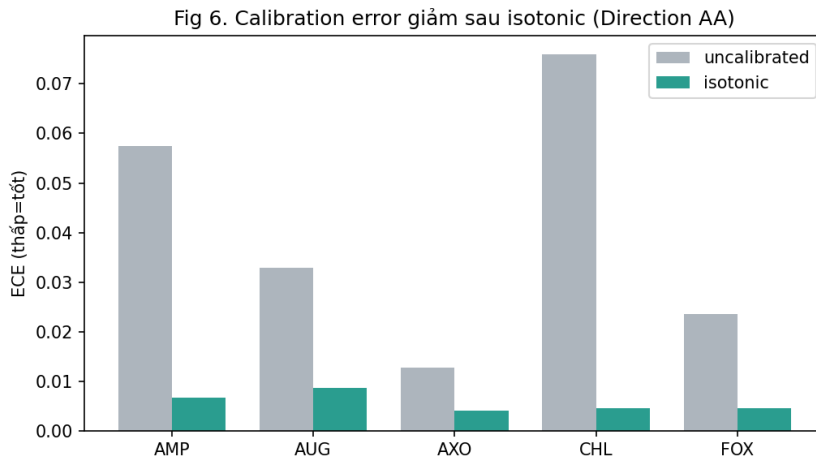


Figure 7.5: ECE decreases markedly after isotonic calibration.

7.9 RQ6: Mutation-level analysis for quinolones

True QRDR mutation calling shows a strong association with resistance: when a mutation is present the resistance rate reaches 0.75–0.88 versus only 0.04–0.15 for wild-type (Figure 7.6). More importantly, the mutation features (only four positions) generalize far better than annotation features under the MLST split: for NAL, F1 holds at 0.85 while annotation features collapse to 0.37; for CIP, F1 reaches 0.56 (other mechanisms such as *qnr*/efflux lie outside the QRDR). This directly demonstrates that mechanism features are superior to annotation features in causality and generalization.

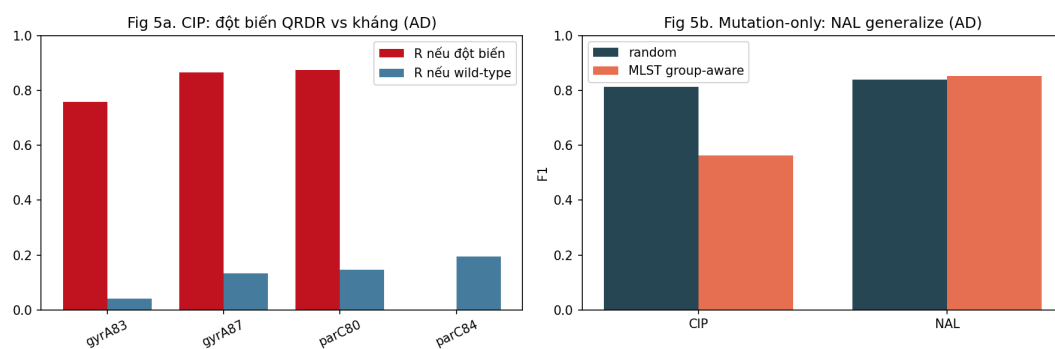


Figure 7.6: Left: QRDR mutations are strongly associated with CIP resistance. Right: mutation features let NAL generalize across lineages.

7.10 Comparison with the original study

Table 7.6 summarizes how this research relates to the original study that introduced the expert-marker dataset [1]. The two share the same data and task, but this research extends the original along several axes: an adaptive per-drug feature-fusion method instead of one fixed feature set; five additional external drugs from BV-BRC; repeated cross-validation with a paired significance test; lineage-aware evaluation using core SNPs and MLST; probability calibration and a decision-curve analysis for deployability; and real QRDR mutation calling for the quinolone group. The contribution is therefore not only a competitive predictor but also a more complete and better-validated study.

Table 7.6: What this research adds relative to the original study.

Aspect	Original study	This research
Feature approach	Fixed 50 expert markers	Adaptive per-drug fusion (many modules)
Antibiotics	5-drug dataset	5 original + 5 external (BV-BRC)
Evaluation	Single split	Repeated CV + paired t-test
Lineage check	Not reported	Lineage-aware split (core SNP / MLST)
Deployability	Not assessed	Calibration + decision curve
Mechanism	Gene presence/absence	+ real QRDR mutation analysis

Overall, this chapter shows a consistent picture: the accessory genome is the dominant signal; adaptive feature fusion is competitive and improves F1 on 4/5 drugs while selecting a suitable representation per drug; the additional lineage-aware, external, calibration and mutation analyses make clear which signals are mechanism-driven and robust, and which drugs need more diverse data. Chapter 8 draws conclusions and suggests future work.

CHAPTER 8. CONCLUSIONS

8.1 Summary

This research has built and thoroughly evaluated an antibiotic-aware adaptive feature fusion framework for AMR prediction in *Salmonella*. The problems solved include: confirming that the accessory genome is the dominant predictive signal, outperforming core SNPs; proposing an adaptive per-drug feature-fusion method that reaches high performance and improves F1 on 4 of the 5 drugs over the strong expert-marker baseline, with each antibiotic selecting a different best module; adding lineage-aware evaluation through true phylogenetic splits that identifies antibiotics with a robust mechanism signal (AXO, FOX, TET) and those that are more lineage-dependent (AMP, AUG, CHL, CIP, NAL, GEN); and performing true QRDR mutation calling (*gyrA*, *parC*) that provides lineage-robust, mechanism-based features for the quinolone group. Compared with the original study, this research adds external multi-drug validation, paired statistical testing, lineage-aware evaluation, calibration, and mutation analysis. The models are well-calibrated and useful for screening.

The research also has remaining limitations. The original dataset contains only five antibiotics and is almost single-clonal (about 97%), so it is not diverse enough to claim generalization across many lineages/serovars. The external validation uses genome annotation features rather than a complete sequence pipeline, and mutation calling is limited to the QRDR of *gyrA/parC*. The results should therefore be read as a competitive and carefully validated proof-of-concept, with the lineage-dependent drugs needing more diverse data before deployment on new lineages.

8.2 Suggestion for Future Works

Future directions include: extending mutation calling to *gyrB/parE*, *qnr* and efflux genes to explain the part of CIP resistance outside the QRDR; integrating sequence-based annotation via CARD/ResFinder for the most important accessory features to increase interpretation coverage; collecting more lineage/serovar-diverse data to test true generalization; and refactoring the code into a reproducible library with automated tests, benchmarked against rule-based baselines. These directions would raise the research from a proof-of-concept to a more trustworthy research pipeline.

REFERENCE

- [1] Y. You, K. Tan, Z. Jiang **and** L. Zhang, “Developing a predictive platform for Salmonella antimicrobial resistance based on a large language model and quantum computing,” *Engineering*, **jourvol** 48, **number** 5, **pages** 174–184, 2025. DOI: 10.1016/j.eng.2025.01.013.
- [2] C. J. Murray **and others**, “Global burden of bacterial antimicrobial resistance in 2019: A systematic analysis,” *The Lancet*, **jourvol** 399, **number** 10325, **pages** 629–655, 2022.
- [3] M. Nguyen **and others**, “Using machine learning to predict antimicrobial minimum inhibitory concentrations and associated genomic features for nontyphoidal salmonella,” *Journal of Clinical Microbiology*, **jourvol** 57, **number** 2, e01260–18, 2019.
- [4] S. G. Earle **and others**, “Identifying lineage effects when controlling for population structure improves power in bacterial association studies,” *Nature Microbiology*, **jourvol** 1, **number** 5, **pages** 1–8, 2016.
- [5] V. Bortolaia **and others**, “Resfinder 4.0 for predictions of phenotypes from genotypes,” *Journal of Antimicrobial Chemotherapy*, **jourvol** 75, **number** 12, **pages** 3491–3500, 2020.
- [6] B. P. Alcock **and others**, “Card 2023: Expanded curation, support for machine learning, and resistome prediction at the comprehensive antibiotic resistance database,” *Nucleic Acids Research*, **jourvol** 51, **number** D1, **pages** D690–D699, 2023.
- [7] E. Zankari, R. Allesøe, K. G. Joensen, L. M. Cavaco, O. Lund **and** F. M. Aarestrup, “Pointfinder: A novel web tool for wgs-based detection of antimicrobial resistance associated with chromosomal point mutations in bacterial pathogens,” *Journal of Antimicrobial Chemotherapy*, **jourvol** 72, **number** 10, **pages** 2764–2768, 2017.
- [8] L. S. Redgrave, S. B. Sutton, M. A. Webber **and** L. J. Piddock, “Fluoroquinolone resistance: Mechanisms, impact on bacteria, and role in evolutionary success,” *Trends in Microbiology*, **jourvol** 22, **number** 8, **pages** 438–445, 2014.
- [9] A. J. Page, C. A. Cummins, M. Hunt **and others**, “Roary: Rapid large-scale prokaryote pan genome analysis,” *Bioinformatics*, **jourvol** 31, **number** 22, **pages** 3691–3693, 2015.
- [10] M. C. Maiden **and others**, “Multilocus sequence typing: A portable approach to the identification of clones within populations of pathogenic microorganisms,”

- Proceedings of the National Academy of Sciences*, **jourvol** 95, **number** 6, **pages** 3140–3145, 1998.
- [11] F. Pedregosa **and others**, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, **jourvol** 12, **pages** 2825–2830, 2011.
- [12] A. J. Vickers **and** E. B. Elkin, “Decision curve analysis: A novel method for evaluating prediction models,” *Medical Decision Making*, **jourvol** 26, **number** 6, **pages** 565–574, 2006.
- [13] R. D. Olson **and others**, “Introducing the bacterial and viral bioinformatics resource center (bv-brc): A resource combining patric, ird and vipr,” *Nucleic Acids Research*, **jourvol** 51, **number** D1, **pages** D678–D689, 2023.

APPENDIX

A. REPRODUCIBILITY AND ARTIFACTS

This appendix describes the components that allow the results of the research to be reproduced.

A.1 Data Sources

The original set is taken from a public repository¹ (accessory gene matrix, core SNPs, and S/R labels for five antibiotics). The external set is self-queried from BV-BRC [13]: S/R labels from the AMR metadata table, functional gene-family features and the MLST type via the API. No raw reads are required.

A.2 Notebook Catalogue

The code is organized into notebooks numbered in logical order. Table A.1 lists the main notebooks.

Table A.1: Main notebooks and their roles.

Notebook	Role
03_signal_source	Accessory genome versus core SNPs
14_adaptive_fusion	Adaptive feature fusion on the five original antibiotics
16_new_drug	New-drug simulation (leave-one-drug-out)
20_stat_test	Statistical testing (simple modules)
21_stat_test_fusion	Statistical testing (full modules)
22_annotation	Biological annotation of top features
23_calibration	Probability calibration and screening utility
24_lineage	Phylogenetic-lineage evaluation (core SNP)
25_external_mlst	External real-prevalence + MLST group-aware
26_mutation	QRDR mutation calling for gyrA/parC

A.3 Environment and Execution

The notebooks run on Google Colab or a Python environment with `numpy`, `pandas`, `scikit-learn` [11], `xgboost`, `scipy` and `requests`. Each notebook downloads the required data itself, performs feature selection on the training set only, and saves results as CSV files. The tables and figures in Chapter 7 are generated directly from these CSV files.

¹<https://github.com/347251369/Antimicrobial-resistance-prediction-in-Salmonella>

B. SUPPLEMENTARY RESULTS

This appendix provides additional detailed results supplementing Chapter 7.

B.1 Population Structure from Core SNPs

Table B.1 shows the number of lineages and the size of the largest lineage at different SNP-distance thresholds, evidencing that the dataset is almost single-clonal.

Table B.1: Population structure (core SNP clustering).

SNP-distance threshold	No. of lineages	Largest lineage (%)
0.05	14	97.4
0.10	13	97.5
0.20	13	97.5
0.30	8	99.1

B.2 QRDR Mutation Association with Resistance

Table B.2 lists the resistance rate by mutation status at the QRDR positions for CIP and NAL.

Table B.2: QRDR mutations and resistance rate (R-rate).

Drug	Position	R-rate if mutated	R-rate if wild-type
CIP	gyrA83	0.76	0.04
CIP	gyrA87	0.87	0.13
CIP	parC80	0.88	0.15
NAL	gyrA83	0.77	0.06
NAL	gyrA87	0.84	0.04
NAL	parC80	0.75	0.08